



UNIVERSIDADE FEDERAL DE MINAS GERAIS
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

TRABALHO PRÁTICO: PokéCBattles

Daniel Costa Andrade

Julho de 2026

1. Introdução	3
2. Descrição do Algoritmo e Procedimentos utilizados	3
Exemplo de Estilo de Texto Esperado:	3
Exemplo de Execução:	4
3. Testes e Erros	4
Processo de Criação do Algoritmo	4
Exemplos de Erros Comuns e Correções	5
Testes Realizados	5

Introdução

O trabalho consiste em um jogo de batalha clássico de Pokémon entre dois treinadores, baseado em turnos. Para ganhar o jogo e finalizá-lo, é necessário que um treinador nocauteie todos os Pokémons do outro treinador. Os turnos funcionam da seguinte forma: O primeiro treinador ataca o Pokémon do outro treinador e após finalizar o ataque, o turno alterna e o treinador que estava defendendo, agora ataca.

O mecanismo de batalha depende do tipo dos Pokémons na batalha e seus atributos. Cada tipo elemental tem vantagem sobre outro tipo e desvantagem sobre outro, logo, se o tipo do Pokémon atacante tiver vantagem sobre o defensor, ele causa 20% de dano à mais e se tiver desvantagem, causa 20% à menos.

Agora, sobre o programa. A entrada é dada por um arquivo txt contendo número de Pokémons por treinador, nome de cada um e seus atributos (valor de ataque, defesa e vida, e tipo). Já a saída, é progresso de nocaute de cada Pokémon (dizendo qual Pokémon nocauteou qual), a lista de todos que foram nocauteados e o treinador vencedor.

Eu acredito que o objetivo final do trabalho seja tornar todo o aprendizado do semestre em um código sucinto, que no final, representa em um jogo, toda a diversão que tive aprendendo programação nesse primeiro semestre.

Descrição do Algoritmo e Procedimentos utilizados

O programa começa utilizando **struct** para facilitar o agrupamento dos atributos necessários para cada Pokémon, contendo: nome[MAX_NOME] (nome do pokedex com tamanho máximo definido previamente pela constante (MAX_NOME = 50), ataque (float), defesa (float), vida (float), tipo[MAX_TIPO] (tipo elemental com tamanho máximo definido previamente pela constante (MAX_TIPO = 20) e derrotado (booleano para identificar o estado de batalha do Pokémon).

Em seguida, é criada a função **obtertipo** para calcular o dano causado de acordo com a afinidade ou oposição ao tipo elemental do(s) Pokémon(s). O parâmetro ***tipo_atq** é a string que contém o tipo do Pokémon atacante e ***tipo_def** é a do Pokémon que sofre o ataque. Usei a função **strcmp** da biblioteca **<string.h>** para comparar cada tipo com a entrada. Se a entrada for igual à um determinada tipo, ele vai devolver o dano multiplicado de acordo com a afinidade ou oposição elemental.

Agora, partindo pra **main**, iniciei criando um arquivo txt para inserir os Pokémons e seus atributos. Em seguida, fiz o programa ler as informações contidas no arquivo e preencher os times para cada jogador, criando dois vetores **j1 e j2** para armazenar as **structs** de cada Pokémon. Ele usa os inteiros **n e m** (números de Pokémon da equipe colocados no arquivo) no **for** em seguida para ir lendo os atributos de cada Pokémon até que acabe todos e depois, fecha o arquivo.

Começando o combate, tem os seguintes atributos:

p1 e p2: São os índices que mostram qual Pokémon está na arena. Começam em 0 (o primeiro de cada equipe).

while (p1 < n && p2 < m): O combate acontece enquanto os dois treinadores ainda tiverem Pokémon disponíveis, se p1 chegar a **n** ou p2 chegar a **m**, significa que a equipa de um deles acabou e o ciclo termina.

Beleza, agora, começa os turnos. O programa chama a função **obtertipo** enviando o tipo do atacante (**j1[p1]**) e do defensor (**j2[p2]**). Logo depois, ele calcula o **poder_ataque** multiplicando o ataque base pelo tipo. Se o poder de ataque for maior que a defesa do oponente, o dano é a diferença, caso contrário, o dano é normal.

Depois do turno, agora se verifica se teve nocautes durante o turno. Se a vida do treinador defendendo cair para zero ou menos, ele é marcado como **derrotado = true** e logo em seguida exibindo quem venceu o duelo, e o índice p2 aumenta (**p2++**), trazendo o próximo Pokémon do outro treinador. O turno não muda, o que significa que o Pokémon vencedor atacará primeiro no próximo ciclo.

Quando acaba a batalha, o programa sai do loop do **while** e vai se encerrando. Ele começa verificando quem ganhou a batalha, se **p1 < n**, significa que o Treinador 1 ainda tinha Pokémon vivos quando o ciclo acabou, então, ele ganhou a partida. Depois, o programa faz uma varredura completa nos dois vetores usando **for**. Se o **atributo .derrotado** for falso (**j1[i].derrotado**), o nome do Pokémon é impresso na lista de sobreviventes. A variável **primeiro** dentro do **if** serve apenas para controlar os espaços, ela garante que o primeiro nome impresso não tenha um espaço antes dele, mantendo a formatação limpa.

Exemplo de Execução:

Adicione um exemplo de input e descreva como seu código é executado em cada passo.

Input:

3 2

Squirtle 10 15 15 água

Vulpix 15 15 15 fogo

Onix 5 20 20 pedra

Golem 20 5 10 pedra

Charmander 20 15 12 fogo

Processo:

1. O programa abre ***dados.txt***
2. Executa a leitura de ***n = 3*** e ***m = 2***, imprime os valores e, em seguida, os atributos dos 5 Pokémon conforme lê cada linha
3. A batalha inicia: ***Squirtle (Treinador 1, p1=0)*** contra ***Golem (Treinador 2, p2=0)***. ***Squirtle*** ataca primeiro por ser o primeiro turno. Tipo água contra pedra dá vantagem de 1.2x no ataque. Ataque modificado = $10 * 1.2 = 12$. Como $12 > 5$ (defesa do Golem), o dano é $12 - 5 = 7$. A vida do Golem vai de 10 para 3.
4. Turno 2: ***Golem*** ataca ***Squirtle***. Tipo pedra contra água não tem modificador. Ataque = 20. Defesa de Squirtle = 15. Dano = $20 - 15 = 5$. Vida do Squirtle cai para 10.
5. Turno 1: ***Squirtle*** ataca ***Golem***. Dano de 7 é aplicado novamente. Vida do ***Golem*** cai para -4. ***Golem*** é nocauteado. Imprime: ***Squirtle venceu Golem***. O índice ***p2*** sobe para 1 (***Charmander*** entra).
6. Os turnos prosseguem seguindo estritamente as mesmas regras matemáticas de dano e vantagem de tipos até que o time do treinador 2 acabe.

Output:

3 2

Squirtle 10 15 15 água

Vulpix 15 15 15 fogo

Onix 5 20 20 pedra

Golem 20 5 10 pedra

Charmander 20 15 12 fogo

Squirtle venceu Golem

Charmander venceu Squirtle

Vulpix venceu Charmander

Treinador 1 venceu

Pokemons sobreviventes:

Vulpix Onix

Pokemons derrotados:

Squirtle Golem Charmander

Testes e Erros

Os testes (tirando os que exemplifiquei na sessão “**Testes Realizados**”) usados foram os inseridos pelo professor junto com os arquivos do trabalho. Já os erros, foram exemplificados mais pra frente nesse documento aqui.

Processo de Criação do Algoritmo

O desenvolvimento começou estruturando a **struct** do Pokémon. O primeiro problema real foi controlar os turnos sem colocar aquele monte de código repetido e gigantesco. A salvação foi usar uma variável de controle de estado (**turno**) e índices simples (**p1 e p2**) para gerenciar qual criatura está ativa na arena, evitando repetição desnecessária de código. O que foi mais trabalhoso mesmo foi parar de ler do terminal e passar a ler de arquivo, porque deu uma dor de cabeça ajustar os escopos de leitura e tratar os ponteiros nulos (**FILE ***) para o código não dar **segmentation fault**.

Exemplos de Erros Comuns e Correções

Erro: (Divisão de Inteiros / Arredondamento) Inicialmente, os atributos de ataque e modificadores foram tratados puramente como valores inteiros (*int*). Isso gerava problemas pois o multiplicador de tipo de 20% aplicado a um ataque de valor 15 resultava em truncamento matemático incorreto da fração na memória ($15 * 1.2 = 18.0$, mas em casos ímpares perdia-se precisão).

1. Correção: Alterei a tipagem dos atributos ofensivos, defensivos e de pontos de saúde dentro da struct para dados do tipo ponto flutuante (float), realizando a formatação de exibição sem casas decimais (%.0f) apenas no momento do output.

Erro: (Loop Infinito por Nocaute) Quando um Pokémon era nocauteado, o programa passava para o próximo membro do time incrementando o índice, porém esquecia de pular a alternagem de turno comum. Isso fazia com que o Pokémon recém-chegado atacasse imediatamente na mesma rodada sem respeitar a vez do oponente que havia acabado de garantir o nocaute.

2. Correção: Adicionei uma estrutura condicional **if/else** no final do bloco do laço principal. Agora, se tiver um nocaute (vida ≤ 0), o índice é incrementado mas a alternagem do turno é ignorada e a inversão da variável turno só ocorre se o Pokémon defensor resistir ao golpe recebido.

Testes Realizados

Teste com Arquivo Inexistente: Execução do programa sem a presença do arquivo dados.txt no diretório de destino. O programa disparou perfeitamente a mensagem de tratamento de erro programada e abortou a execução sem gerar falhas de **segmentation fault**.

Teste de Dano Mínimo: Criação de um cenário de teste onde um Pokémon tinha ataque modificado de valor 5 contra um oponente de defesa de valor 30. O programa validou com sucesso a estrutura condicional aplicando a diminuição exata de 1.0 ponto de vida por turno, impedindo a ocorrência de cura acidental (dano negativo) ou dano nulo.

4. Conclusão

Achei o trabalho muito pertinente com o que vi em sala de aula. Acredito que esse trabalho serviu pra eu fixar tudo aquilo que aprendi e me tornar ainda mais completo na programação em C. Aprendi a lidar melhor com loops, ponteiros, arquivos e o uso

de struct. Se eu tivesse que fazer o mesmo trabalho de novo, não acho que mudaria nada nele, pois a minha experiência individual com esse trabalho foi excelente.